

Approval-Based Signup Flow - Production Plan

Prepared as a senior SaaS architecture recommendation for replacing immediate account creation with a reviewed signup workflow.

Recommendation: Use a separate SignupRequest table as the intake layer. Do not create real User or Organization records until a Super Admin approves the request.

This keeps pending prospects separate from real customers, avoids fake organizations, prevents accidental dashboard access, simplifies billing, and gives the business a clean audit trail.

Target Flow

1. Visitor submits the signup form.
2. The system saves a pending SignupRequest.
3. The visitor sees the success modal: "Thank you for registering. Our team will contact you after reviewing your request."
4. The system notifies Super Admins.
5. A Super Admin approves or rejects the request.
6. Only after approval, the system creates the organization, first Admin user, required authentication records, default sequences/settings, and subscription or trial.
7. The approved contact receives an activation email.
8. The contact sets a password or verifies their email, then logs in manually.

Why A Separate SignupRequest Table Is Best

A signup request is not the same thing as a user account. It is a business intake record. Keeping it separate is the cleanest production model.

- No inactive or fake organizations polluting customer data.
- No unapproved users in authentication tables.
- No accidental access to dashboard, billing, sequences, or tenant data.
- Clear Super Admin review workflow.
- Easier duplicate handling, expiry, rejection, and audit logging.
- Cleaner reporting because real customer tables contain only approved accounts.

SignupRequest Fields

Business And Contact

```
id
company_name
company_domain
contact_name
contact_email
contact_phone
requested_plan
```

company_size
message / use_case

Status And Review

status: pending | approved | rejected | expired | cancelled
approved_by
approved_at
rejected_by
rejected_at
rejection_reason_internal
rejection_reason_public

Normalization And Duplicate Checks

normalized_email
normalized_company_name
normalized_domain

Security And Tracking

ip_address
user_agent
source
utm_source
utm_campaign
expires_at
created_at
updated_at

Created Records After Approval

created_organization_id
created_user_id

Audit Log Table

Add a separate event table for compliance, support, and debugging.

signup_request_events.id
signup_request_events.signup_request_id
signup_request_events.actor_type: system | super_admin
signup_request_events.actor_id
signup_request_events.action: created | notified | approved | rejected | expired |
activation_sent
signup_request_events.metadata
signup_request_events.created_at

Required APIs

Public API

POST /signup-requests

This endpoint creates a pending request and always returns a generic success response. It must not log the visitor in or redirect them to the dashboard.

Super Admin APIs

```
GET /admin/signup-requests
GET /admin/signup-requests/:id
POST /admin/signup-requests/:id/approve
POST /admin/signup-requests/:id/reject
```

Optional Admin APIs

```
POST /admin/signup-requests/:id/resend-activation
POST /admin/signup-requests/:id/mark-contacted
```

Activation APIs

```
GET /activate?token=...
POST /activate
```

Super Admin Approval Flow

Approval should happen in one database transaction.

1. Lock the SignupRequest row.
2. Confirm the status is still pending.
3. Check for duplicate active user, organization, company, or domain.
4. Create the organization.
5. Create the first Admin user.
6. Create the auth identity or activation token record.
7. Create default sequences, settings, roles, and tenant defaults.
8. Create subscription or trial records.
9. Mark the signup request as approved.
10. Store created_organization_id and created_user_id.
11. Write audit events.

Send emails only after the transaction commits, preferably through an outbox table or background job.

Super Admin Rejection Flow

1. Lock the request.
2. Confirm it is still pending.
3. Set status to rejected.
4. Store the rejecting Super Admin, timestamp, and reason.
5. Write an audit event.
6. Optionally send a polite rejection email.

Do not immediately delete rejected requests. Keep them for audit history and duplicate handling.

Email And Activation Link Flow

The best approach is to avoid collecting a password during the initial signup request. After approval, send an activation link.

- The link should be single-use.
- The link should expire, usually after 24 to 72 hours.
- Only a hashed token should be stored in the database.
- The token should be invalidated after use.
- The page should let the approved user set a password and verify their email.
- After activation, the user logs in manually.

Duplicate Handling

Normalize email, company, and domain before comparison.

```
lowercase(trim(email))
lowercase(trim(company_name))
lowercase(domain)
```

Recommended Rules

- If the same email already has a pending request, do not create a duplicate. Show the same generic success modal.
- If the same email is already an active user, do not reveal that publicly. Return the same generic response or route to login/reset password separately.
- If the same company or domain has a pending request, flag it for Super Admin review or merge internally.
- If the company already exists, block automatic approval unless a Super Admin explicitly links the user to the existing organization.

Database Protection

- Use database constraints in addition to application checks.
- Unique pending request by normalized_email.
- Unique active user email.
- Unique active organization domain if domain is reliable for your business model.

Security Requirements

- Public signup response should always be generic.
- Do not reveal whether an email or company already exists.
- Rate-limit signup requests by IP and email.
- Add CAPTCHA if spam or abuse becomes likely.
- Store IP address and user agent.
- Never store plain activation tokens.
- Never store raw passwords in SignupRequest.
- Restrict approval and rejection APIs to Super Admin only.
- Use CSRF protection if admin authentication uses cookies.
- Validate all input server-side.
- Use audit logs for every important decision.
- Use an outbox or background job for email delivery.
- Expire old pending requests, for example after 30 days.

Expiry And Cleanup

- Set expires_at when the request is created.
- Automatically mark old pending requests as expired.
- Keep rejected and expired requests for a defined retention period.
- Use retention rules that match company policy and applicable privacy requirements.

Race-Condition Handling

Use row locking and database constraints so two Super Admins cannot approve the same request twice.

- Use SELECT ... FOR UPDATE or the ORM equivalent.
- Approval should be idempotent where possible.
- If already approved, return the existing created user and organization.
- If rejected or expired, block approval unless the request is explicitly reopened.
- If creation fails, roll back the transaction.
- Use an outbox table so email retries do not create duplicate accounts.

Final Recommendation

Use SignupRequest as the only record created during public signup. Create real SaaS tenant data only after Super Admin approval.

SignupRequest pending -> Super Admin approves -> Organization/User/Auth/Subscription created -> Activation email sent -> User sets password -> User logs in manually

This approach is safer, cleaner to audit, easier to support, and better aligned with a controlled B2B SaaS onboarding model.